Universidade Estadual de Campinas
Instituto de Computação

Tarefa 5

Aventuras pelo mapa

MC322 - Programação Orientada a Objetos

Professor: Marcelo da Silva Reis
PEDs: Caroline Nakazato, Giovanne Mariano, Gustavo Morais e Marcos Freire
PADs: Caio, Lia, Paulo e Vinicius

---

1. Descrição Geral

Até este ponto da disciplina, você desenvolveu um sistema de combate relativamente completo inspirado em *Slay the Spire*, incluindo cartas, efeitos, gerenciamento de turnos e organização do projeto com ferramentas como Gradle e Javadoc.

No entanto, o jogo ainda está limitado a batalhas isoladas. Em jogos do gênero *roguelike deckbuilder*, como *Slay the Spire*, uma partida completa envolve a progressão por uma sequência de desafios, nos quais o jogador toma decisões que impactam diretamente o desenrolar da campanha.

Neste laboratório, você irá evoluir o projeto para incluir um **sistema de progressão**, permitindo que o jogador navegue entre múltiplas batalhas. O estado do jogador — como vida e baralho — deve ser mantido ao longo dessa progressão, aproximando o jogo de uma experiência mais completa.

Do ponto de vista de modelagem, essa progressão pode ser representada de diversas formas. Uma abordagem recomendada é modelar o mapa do jogo como uma **estrutura em árvore**, na qual cada nó representa uma batalha e as conexões representam as escolhas disponíveis ao jogador. Essa estrutura permite explorar conceitos importantes de organização de dados e controle de fluxo.

Além da expansão do jogo, este laboratório introduz uma prática fundamental no desenvolvimento de software: a **escrita de testes automatizados**. À medida que sistemas crescem em complexidade, torna-se cada vez mais importante garantir que funcionalidades existentes continuem funcionando corretamente. Para isso, você deverá implementar testes automatizados para partes relevantes do seu código.

Assim como nos laboratórios anteriores, você tem liberdade para expandir e adaptar seu projeto, podendo utilizar quaisquer recursos da linguagem Java, incluindo bibliotecas externas gerenciadas via Gradle.

2. Objetivos

Ao final deste laboratório, o aluno deverá ser capaz de:

- Modelar e implementar um sistema de progressão entre múltiplas batalhas;
- Utilizar estruturas de dados para representar escolhas do jogador (ex: árvore);
- Implementar testes unitários utilizando JUnit;
- Compreender o papel da cobertura de testes na validação de software;
- Aplicar boas práticas de desenvolvimento, incluindo organização, documentação e uso de controle de versão;
- Expandir e evoluir um sistema existente de forma organizada e incremental.

3. Atividades

Esta seção descreve as atividades do laboratório. O objetivo é evoluir o jogo para suportar progressão entre múltiplas batalhas, introduzir testes automatizados e explorar persistência de estado.

Assim como nos laboratórios anteriores, as instruções abaixo definem os requisitos mínimos esperados. Você é livre para adaptar e expandir sua implementação, desde que respeite os objetivos propostos.

As seguintes diretrizes gerais devem ser observadas:

- **Princípios de POO:** Continue utilizando herança, polimorfismo e encapsulamento para organizar seu código. Evite duplicação desnecessária e favoreça reutilização;
- **Organização do Projeto:** Mantenha o projeto estruturado de forma clara dentro do padrão do Gradle;
- **Liberdade Técnica:** Você pode utilizar qualquer recurso da linguagem Java, incluindo bibliotecas externas adicionadas via Gradle;
- **Autonomia:** Algumas decisões de modelagem são intencionalmente deixadas em aberto. Escolha soluções coerentes com o restante do seu projeto.

3.1. Sistema de Mapa e Progressão

Figura 1: O mapa em Slay the Spire. Veja que ele é organizado como uma árvore em que cada nó representa um combate, loja ou evento genérico.

O jogo deve deixar de ser uma batalha isolada e passar a permitir **progressão entre múltiplas batalhas**. O sistema de progressão deve funcionar da seguinte forma:

- O jogo deve ser representado por um mapa com vários nós. Este mapa deve ser mantido entre batalhas;
- O jogador inicia em um nó raíz deste mapa;
- No começo do jogo e após cada batalha vencida, o jogador deve escolher um **nó adjacente e não visitado** para o qual progredir;
  - Cuidado para que o jogador não fique preso em uma ponta do mapa. Em Slay the Spire, o jogador só pode ir para nós mais profundos, ou seja, sempre para frente, evitando que ele fique preso.
- Por enquanto, todos os nós representam batalhas distintas;
- A vida e o baralho do jogador são mantidos entre batalhas, mas elementos como efeitos e energia reiniciam;
- O jogo termina em derrota quando a vida do jogador chega a zero, ou em vitória quando ele alcança um nó final.

3.1.1. Classe `Batalha`

Implemente uma classe `Batalha` responsável por encapsular a lógica de um combate individual.
Essa classe pode, por exemplo:

- Receber o jogador e os inimigos envolvidos;
- Executar o combate até seu término;
- Retornar o resultado da batalha (vitória ou derrota).

A forma exata de implementação fica a seu critério, mas a classe deve permitir reutilização dentro do sistema de mapa.

3.1.2. Representação do Mapa

Recomenda-se representar o mapa como uma **árvore**, na qual cada nó contém uma batalha e referências para seus próximos nós.

Você pode implementar isso de diferentes formas:

- Utilizando estruturas da biblioteca padrão, como `DefaultMutableTreeNode`;
- Utilizando bibliotecas externas;
- Implementando sua própria estrutura de árvore (ex: classe `NoMapa` com lista de filhos).

A escolha da estrutura faz parte da atividade. Independentemente da abordagem escolhida, o jogador deve ser capaz de navegar pelo mapa por meio de escolhas no terminal.

3.2. Testes Automatizados

À medida que sistemas crescem, torna-se importante garantir que funcionalidades continuem funcionando corretamente. Para isso, utilizamos testes.

3.2.1. Visão Geral

Testes de software podem ser classificados de diferentes formas:

- **Abordagem:**
  - Caixa branca — considera a implementação interna (ou seja, leva em consideração o código);
  - Caixa preta — considera apenas entradas e saídas (ou seja, não leva em consideração o código);
  - Testes manuais — executados por humanos.
- **Tipo:**
  - Testes funcionais — verificam comportamento esperado;
  - Testes não funcionais — avaliam aspectos como desempenho, usabilidade e etc.
- **Automatizados:**
  - Testes unitários — testam pequenas partes isoladas do código;
  - Testes de integração — verificam interação entre componentes;
  - Testes de sistema — avaliam o sistema como um todo.

Neste laboratório, o foco será em **testes unitários automatizados**.

3.2.2. JUnit

Utilize o **JUnit** para implementar testes automatizados no seu projeto. O JUnit pode ser facilmente integrado via Gradle.

Os testes devem ser colocados no diretório:

```
src/test/java
```

3.2.3. Cobertura de Código

A avaliação dos testes será baseada em **cobertura de código**, ou seja, o quanto do seu código é executado pelos testes.

Embora existam diferentes tipos de cobertura, neste laboratório utilizaremos uma medida simples baseada em execução de linhas.

Seu projeto deve atingir pelo menos **40 % de cobertura de código**. Para medir a cobertura, deve ser utilizada alguma biblioteca como JaCoCo (Java Code Coverage) que é facilmente adicionável via gradle.

3.2.4. Exemplo de Teste

A seguir, apresentamos um exemplo um pouco mais completo de método e seus testes, considerando múltiplos cenários possíveis.

Classe:

```
public class Heroi {
    private int vida;
    private int escudo;

    public Heroi(int vida, int escudo) {
        this.vida = vida;
        this.escudo = escudo;
    }

    public void receberDano(int dano) {
        if (this.escudo >= dano) {
            this.escudo = this.escudo - dano;
        } else {
            int danoRestante = dano - this.escudo;
            this.escudo = 0;
            this.vida = this.vida - danoRestante;
        }

        if (this.vida < 0) {
            this.vida = 0;
        }
    }

    public int getVida() {
        return this.vida;
    }

    public int getEscudo() {
        return this.escudo;
    }
}
```

Testes:

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class HeroiTest {

    @Test
    public void danoAbsorvidoPorEscudo() {
        Heroi h = new Heroi(10, 5);
        h.receberDano(3);
        assertEquals(10, h.getVida());
        assertEquals(2, h.getEscudo());
    }

    @Test
    public void danoParcialComEscudo() {
        Heroi h = new Heroi(10, 3);
        h.receberDano(5);
        assertEquals(8, h.getVida());
        assertEquals(0, h.getEscudo());
    }

    @Test
```

```
    public void danoSemEscudo() {
        Heroi h = new Heroi(10, 0);
        h.receberDano(4);
        assertEquals(6, h.getVida());
        assertEquals(0, h.getEscudo());
    }

    @Test
    public void vidaNaoFicaNegativa() {
        Heroi h = new Heroi(3, 0);
        h.receberDano(10);
        assertEquals(0, h.getVida());
    }
}
```

Observação sobre cobertura:
Cada teste acima exercita um caminho diferente dentro do método `receberDano`:

- `danoAbsorvidoPorEscudo`: cobre o caso em que o escudo é suficiente para absorver todo o dano;
- `danoParcialComEscudo`: cobre o caso em que parte do dano atinge a vida;
- `danoSemEscudo`: cobre o caminho em que não há escudo;
- `vidaNaoFicaNegativa`: cobre a condição final que impede a vida de se tornar negativa.

Isso ilustra como diferentes testes são necessários para cobrir os diferentes fluxos de execução de um método. Quanto mais caminhos relevantes do código forem exercitados, maior tende a ser a cobertura de testes.

3.3. Desafio Extra (+0,5 ponto)

Como desafio extra, implemente a persistência do estado do jogo utilizando **JSON**.
A ideia é permitir salvar e carregar o progresso do jogador, incluindo informações como:

- Vida do jogador;
- Baralho;
- Posição no mapa.

O jogador deve poder escolher sair do jogo como uma opção do terminal, caso no qual seu progresso será salvo e carregado quando o jogo for iniciado novamente.

3.3.1. Dicas

- Utilize bibliotecas como `Gson` ou `Jackson`, que podem ser adicionadas via Gradle;
- Crie classes ou métodos responsáveis por converter objetos do jogo para JSON e vice-versa;
- Comece salvando estruturas simples (ex: apenas vida e posição) e evolua gradualmente;
- Evite acoplar diretamente a lógica de serialização às classes principais — prefira separar responsabilidades.

4. Avaliação

A nota do laboratório será composta pelos seguintes critérios:

1. **Sistema de Mapa e Progressão (3,0 pontos)**
    - Implementação de um sistema que permita progressão entre múltiplas batalhas;
    - Manutenção do estado do jogador entre batalhas;
    - Navegação funcional no mapa por meio de escolhas do usuário;

- Coerência geral da solução proposta.

2. **Testes Unitários (3,0 pontos)**
   - Implementação de testes utilizando JUnit;
   - Cobertura de código de pelo menos **40 %**;
   - Testes cobrindo diferentes cenários relevantes da aplicação;
   - Organização e clareza dos testes implementados.

3. **Boas Práticas de Desenvolvimento (4,0 pontos)**
   - Cada classe está em seu próprio arquivo `.java`;
   - Código legível, bem formatado e com nomes adequados;
   - Uso de documentação com Javadoc;
   - Uso adequado de controle de versão (branches e commits descritivos);
   - Uso consistente de conceitos de orientação a objetos (herança, encapsulamento, polimorfismo, interfaces, etc.).

4. **Desafio Extra (até +0,5 ponto)**
   - Implementação de salvamento e carregamento do estado do jogo utilizando JSON.

5. **Penalidade por Não Compilação**

   Caso o projeto:
   - Não compile utilizando `./gradlew build`; ou
   - Não execute corretamente utilizando `./gradlew run`;

   será aplicada uma penalidade de **-5,0 pontos** sobre a nota obtida.

   A nota final mínima é **0,0** (não haverá notas negativas).

4.1. Dicas de Boas Práticas

Um código de qualidade não é apenas aquele que funciona corretamente, mas também aquele que é **fácil de entender, manter e modificar**. Ao longo do desenvolvimento, é importante escrever código pensando não apenas na solução imediata, mas também na sua evolução futura.

Existem diversos princípios de construção de software que ajudam a atingir esses objetivos. A seguir, destacamos alguns dos mais relevantes:

- **DRY (Don't Repeat Yourself):** Evite duplicação de código. Se uma mesma lógica aparece em múltiplos lugares, considere abstraí-la em um método ou classe reutilizável.
- **KISS (Keep It Simple, Stupid):** Prefira soluções simples e diretas. Evite complexidade desnecessária, especialmente quando uma solução mais simples resolve o problema de forma adequada. E não se engane, isso não é ser preguiçoso. Muitas vezes chegar a uma solução simples exige mais esforço intelectual.
- **YAGNI (You Aren't Gonna Need It):** Evite implementar funcionalidades que não são necessárias no momento. Focar apenas no que é necessário ajuda a manter o código mais limpo e compreensível.

Outro conjunto importante de diretrizes são os princípios **SOLID**, que tratam de organização, responsabilidade e acoplamento entre classes. Embora tais princípios não sejam exigidos nos laboratórios, sua compreensão pode ajudar a escrever código mais modular e flexível, facilitando para vocês mesmos darem seguimento nas próximas tarefas e no projeto final.

Além disso, é uma boa ideia – e sinal de maturidade do desenvolvedor – utilizar **padrões de projeto** (*design patterns*), que são soluções reutilizáveis para problemas recorrentes no desenvolvimento de software. Esses padrões ajudam a estruturar melhor o código, melhorar a comunicação entre desenvolvedores e evitar soluções improvisadas para problemas já conhecidos. Um exemplo já utilizado nesta disciplina é o padrão *Observer*, que permite desacoplar eventos da lógica que reage a eles. Como referência para estudo desses conceitos, recomenda-se o site refactoring.guru, que apresenta explicações e exemplos práticos de diversos padrões de projetos.

5. Entrega

A submissão é realizada exclusivamente via repositório no **GitHub**. Para isso:

- Gere uma **release (tag)** com a identificação do laboratório, seguindo o formato: `tarefaX-RA1-RA2`;
- Gere uma release no GitHub a partir dessa tag;
- Envie o link da release no Google Classroom.

Caso optem por manter o repositório **privado**, adicionem os PEDs da disciplina como colaboradores:

- Caroline Nakazato – `@CarolineNakazato`
- Giovanne Mariano – `@giolucasd`
- Gustavo Morais – `@moraguma`
- Marcos Freire – `@marcfreir`

Lembrem-se que:

- O prazo de submissão é até o dia **17/04/2026 às 23h59**.
- Entregas atrasadas serão penalizadas em **25 % da nota obtida**. Não serão aceitas entregas atrasadas após 24/04/2026.
- Os horários de laboratório e atendimento podem (e devem) ser utilizados para tirar dúvidas relacionadas tanto à submissão quanto ao desenvolvimento do laboratório.

5.1. Compilação e Execução

Durante a avaliação, os PEDs utilizarão os seguintes comandos, a partir da raiz do repositório, para compilar e executar o projeto:

```
./gradlew build
./gradlew run
```

A classe principal do projeto deve conter o método:

```
public static void main(String[] args)
```

Essa classe será utilizada como ponto de entrada para a execução do programa. Certifique-se também de que ela está corretamente configurada no arquivo de build do Gradle para permitir a execução do comando `run`.

Dessa forma, a compilação e a execução ocorrerão corretamente no ambiente de avaliação.